

20.02.2024г.

adimitriev@fmi...

Лекция 1

- Ангел Димитриев

Доктор Цонков

д-р Йосифов

Всички материали в github.com/AngelD55УП → ООП → СДАООП - парадигма (стил на програмиране)

↳ Програмата се моделира като обекти, които взаимодействат помежду си.

Оценяване:Текущ контрол (ТК)

- К1 30т. → теория
→ задачи
 - К2 40т. → теория
→ задачи
- НА ЛИСТ

• Домашни по 10 точки

(проверява се от асистенти практ.)

Итог: 100 точки

Изпит

Допускане до изпит:

ТК ≥ 25 точки

Изпит задачи - 100 точки

поне ≥ 40т. (лист)

Изпит теория - 100 точки

поне ≥ 50т.

Изпит теория: теглим от 1 до 8 тема - въпрос 1

теглим от 9 до 15 тема - въпрос 2

Събеседване

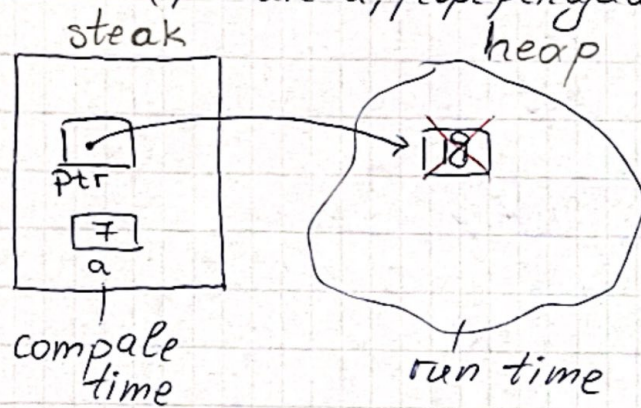
Тройката започва от ≥ 160т. от 300 точки общо

C++ е основополагащ език и използваме него.

Защото е удобно от C++ да започнеш да учиш друг език.

ООП-практикум - присъствие + проект (защита на проект)

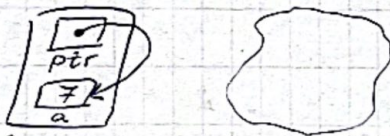
Преговор от УП (указатели, референции, динамична памет)



```
{ int a=7;
  int* ptr = new int (18);
```

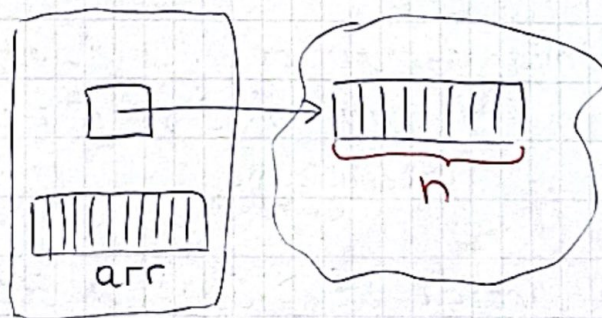
```
delete ptr; - унищожава данните, а не ptr
} a излиза
```

```
{ int a=7;
  int* ptr = &a;
  delete ptr; - използва се само
                  в динамичната памет
}
```



Статични масиви

```
{ int arr[10];
  int n;
  cin >> n;
  int* ptr = new int[n];
  delete [] ptr;
}
```



В общия случай запазваме $n+1$ клетки, защото една клетка ни е за големината на динамическия масив.

```
func(int a) — приемаме a по копие
int a=7;
func(a);
cout << a; //7
```

```
func1(int &a) — приемаме a по референция
a++;
a=10;
func1(a);
cout << a; //11
```

```
func2(int *p) — приемаме a по указател
{ if(!p) // nullptr
  return;
  (*p)--; — дереференциране
}
int x=7;
func2(&x);
cout << x; //6
```

! Разлики между указател и референция

- 1) При референцията не можем да променим адреса
- 2) Референцията трябва да се инициализира

~~int &a;~~ — не може

- 3) Указателят има неутрална стойност (nullptr)

```
int *ptr1 = ...
```

```
char *ptr2 = ...
```

```
cout << ptr1; // отпечатва адрес
```

```
cout << ptr2; // отпечатва символите в паметта
                (приема ptr2 за символен низ)
```

```
ptr1++; // +=4 bytes, защото int е 4 байта
```

```
ptr2++; // +=1 byte, char е 1 байт
```

Namespace

(пространство от имена)

- инструмент за избягване на конфликти от имена
- scope, в който има дефинирани символи

```
namespace ns-name {
    void f() { ... } // функция
    int global = 9; // глобална променлива
```

}

~~f();~~

ns-name::f();

оператор за резолюция

```
using namespace ns-name;
```

f();

global++;

} беге можем да достигнем до нещата

```
using namespace std;
```

вкарва много неща
беге не го ползваме

```
using ns-name::f
f();
```

using std::cout;

using std::cin;

} нямаме namespace, защото е
за конкретна функция

Могат да доведат до конфликти:

```
namespace A {
    f() { ... }
}
```

```
namespace B {
    f() { ... }
}
```

~~int main() {~~

using namespace A;

using namespace B;

f();

} дотук се компилира

// конфликт

Ако направим A::f(); ще работи

Увод в типовете дефинирани
от потребителя.

1. Енумерация

Enum (тип изброен)

def: Тип, рестриктиран до домейн от стойности, които
включват специално дефинирани константи.

Енумератори

```
enum color {  
    red,    // = 0  
    blue,   // = 1  
    orange  // = 2  
}
```

```
color a1 = color::red;  
if (a1 == color::red)
```

...

Всеки енумератор съответства на някакво целочислено
Ако не е указано, то това е предния + 1
Ако на първия не е дадена стойност, той е 0.

! Пример за тема 1:

```
enum t {
```

```
    a,      0
```

```
    b,      1
```

```
    c=5,    5
```

```
    d,      6
```

```
    e=4,    4
```

```
    f#,     5
```

```
    g=(a+b) → това е възможно  
    }      = 1
```

! enum е unscoped.

- „Енумераторите са глобални променливи.“
- Имате имплицитно /неявно/ преобразуване от енумератор към число.

```
enum color {
```

```
    red#,
```

```
    orange
```

```
};
```

```
enum fruit {
```

```
    orange
```

Няма да се компилира,
защото имаме
съвпадения имена
на променливи

```
int x = color::red;
```

Имаме ^{тип color}

```
enum color {
```

```
    red, 0 } стойност
```

```
    orange 1
```

```
};
```

```
enum Animal {
```

```
    dog, 0
```

```
    cat  1
```

```
};
```

```
color c1 = color::orange;
```

```
Animal a1 = Animal::cat;
```

```
if (c1 == a1) { ... } // компилира се
```

1 == 1 → това не трябва, защото са от различни типове

• Enum class — scoped enum

```
enum class color {  
    orange  
};
```

✓ това ще се компилира

```
enum class fruit {  
    orange  
};
```

`int x = color::orange;` // това няма да се компилира
`int x = (int)(color::orange);` // казваме го /преобразуваме и вече работи

✗ `if (c1 == a1)` — няма да се компилира

`sizeof(e1)` — големината на целочислен тип, в който домейн от стойности се „побират“ енумераторите

Кръщаване ги с главни букви (Animal, Color)

Структури

def: Последователност от полета, които се пазят в определен ред. // различава се от масив, защото може да е от различни типове

```
struct Test {  
    int a;  
    char ch[10];  
    bool b;  
};
```

стр → инстанция
класове → обекти

Test t1; ^{инстанция}

t1.a = 10;

strcpy(t1.ch, "ABC"); // имаме 2 масива и това е
разхищение на памет

t1.b = false;

При enum class имаме невяно преобразуване от
енумератор към цяло число.

struct Point

{

int x;

int y;

};

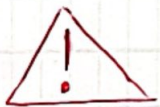
x	y
---	---

В паметта дали ще е масив от
2 елемента или class няма
разлика.

В семантиката има разлика.

Point p{3, 7};

Point p = {3, 7}; // може и с равно



Point *ptr = new Point{3, 4};

delete ptr;

Подаване на инстанции във функции.

≠ (Point p) — копие

≠ (Point &p)

⚠ Ще променям ли обекта/инстанцията?

НЕ → ≠ (const Point &p) // принт функция например

$\neq$ (Point * ptr)

⚠ НЕ $\rightarrow \neq$ (const Point * ptr)

указателен може да се работи, но
не може да променя данните)

double getDist(const Point & t, const Point & s)

void readPoint(Point & p) // промените се отразяват на точката

Влагане на структура

```
struct line {  
    Point beg;  
    Point end;  
};
```

функция, която приема 2 линии и връща дали са
успоредни.

$$\text{slope} = \frac{\Delta y}{\Delta x} = \frac{\text{разликата на } y}{\text{разликата на } x} = \frac{dy}{dx}$$

Абстракция - 1^{ви} OOP принцип

Използваме нещо без да се интересуваме как работи

~~double == double~~ // не можем да ги сравняваме
Когато делим 2 double и вторият е 0 получаваме
безкрайност

Пример за лоша абстракция
struct triangle {

int x1;

int y1;

int x2;

int y2;

int x3;

int y3;

}

// това трябва да са
3 точки

// не ни интересува в какво
пространство

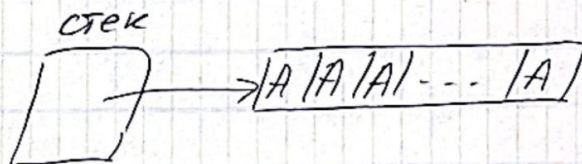
Масиви от инстанции

struct A {};

A arr[10];

|A|A|A|...|A|

A* ptr = new A[n];



delete[] ptr;

// запазваме памет $n \cdot \text{sizeof}(A) + \text{sizeof}(\text{int})$

↑
за голямината
на масива

⚠ $\text{sizeof}(A) \geq \sum_{x \text{ член на } A} \text{sizeof}(x)$

struct B {

char ch; // 1 байт

int x; // 4 байта

};

sizeof(B) = 8

↑

размер на структура

За изпит:

Пример за задачата с N триъгълника:

зар. Въвежда се цяло число N и N триъгълника.
Трябва да ги отпечатаме сортирани по тяхното лице.

Използваме Selection Sort, защото прави минимален брой свар-ове.